

Government Engineering College Kishanganj

Department of Computer Science and Engineering

Computer Networks LAB-05

Text Analyzer Application using TCP Sockets

Objective

To design and implement a client-server application using TCP sockets in C that performs basic text analysis.

Problem Statement

Develop a TCP-based application where:

- The client sends a request in the format:

```
ANALYZE | TEXT
```

- The server processes the text and returns:
 - Number of characters
 - Number of words
 - Number of vowels

Example

Request:

```
ANALYZE | Hello World
```

Response:

```
Chars=11, Words=2, Vowels=3
```

Additional Requirements

- Use TCP sockets (SOCK_STREAM)
- Handle multiple requests until client sends “bye”
- Properly parse input using delimiter |

Server Template

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8081
#define BUFFER_SIZE 1024

int count_words(char *str) {
    int count = 0, in_word = 0;
    -----
    -----
    -----
    return count;
}

int count_vowels(char *str) {
    int count = 0;
    -----
    -----
    -----
    return count;
}

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    char buffer[BUFFER_SIZE];
```

```

// Create Socket ----

// Initialize the value --

// Write bind syntax here -----
//Write Listen Syntax here ----
//Write Accept function call --


while (1) {
    memset(buffer, 0, BUFFER_SIZE);
    int valread = read(new_socket, buffer,
        BUFFER_SIZE);

    if (valread <= 0) break;

    buffer[strcspn(buffer, "\n")] = 0;

    if (strcmp(buffer, "bye") == 0) break;

    char command[20], text[BUFFER_SIZE];
    sscanf(buffer, "%[^|]|%[^\\n]", command, text);

    if (strcmp(command, "ANALYZE") == 0) {
        int chars = strlen(text);
        int words = count_words(text);
        int vowels = count_vowels(text);

        char response[BUFFER_SIZE];
        sprintf(response, "Chars=%d, Words=%d, Vowels=%d", chars, words, vowels);

        send(new_socket, response, strlen(response),
            0);
    } else {
        char *msg = "Invalid Command";
        send(new_socket, msg, strlen(msg), 0);
    }
}

```

```

    }

    close(new_socket);
    close(server_fd);
    return 0;
}

```

Client Template

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8081
#define BUFFER_SIZE 1024

int main() {
    int sock;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE];
    char input[BUFFER_SIZE];

    // Create Socket ----

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr)
        ;

    connect(sock, (struct sockaddr *)&serv_addr, sizeof(
        serv_addr));

    -----
    -----
    -----

```

```
    close(sock);  
    return 0;  
}
```